APB Slave Register Block (apb_slave): Design Specification

Document version 1.0 | Status: released for verification | Module name: `apb_slave`

1. Introduction

1.1 Purpose

This document specifies the behavior of `apb_slave`, a small memory-mapped register block that connects to an AMBA APB bus as a slave. It is the reference for design and verification. Anything not described here is not guaranteed by the design.

1.2 Scope

The block contains fifteen general-purpose read/write registers and one read-only identification register. It supports byte-lane write strobes, an error response, and a configurable number of wait states. It has a single clock and a single active-low asynchronous reset.

1.3 References

- AMBA APB Protocol Specification (APB3 handshake with the APB4 PSTRB and PSLVERR signals). Where this document is silent on a protocol point, the AMBA APB specification applies.

2. Features

- 32-bit data bus, 8-bit address bus.
- 15 read/write 32-bit registers and 1 read-only ID register.
- Byte-granular writes controlled by `PSTRB`.
- `PSLVERR` error response for illegal accesses.
- Parameterizable number of wait states (`WAIT_CYCLES`).
- Parameterizable ID value (`ID_VALUE`).
- Asynchronous active-low reset.

3. Interface

3.1 Parameters

Parameter	Default	Legal range	Description
`WAIT_CYCLES`	1	0 to 255	Number of wait states inserted in the access phase of every transfer. 0 means a zero-wait-state slave.
`ID_VALUE`	0xA9B00001	any 32-bit value	Value returned when the ID register is read.

3.2 Ports

Port	Dir	Width	Description
`pclk`	in	1	Bus clock. All activity is on the rising edge.
`presetn`	in	1	Active-low asynchronous reset.
`psel`	in	1	Slave select, asserted by the master for the duration of a transfer.
`penable`	in	1	Asserted by the master in the second and later cycles of a transfer (access phase).
`pwrite`	in	1	1 = write, 0 = read.
`paddr`	in	8	Byte address.
`pwdata`	in	32	Write data.
`pstrb`	in	4	Write strobes, one bit per byte lane (bit 0 = pwdata[7:0], bit 3 = pwdata[31:24]).
`prdata`	out	32	Read data.
`pready`	out	1	Slave ready. Low extends the access phase (wait state). High completes the transfer.
`pslverr`	out	1	Slave error, qualified by `pready`.

4. Functional Description

4.1 Transfer protocol

Every transfer has two phases:

- **Setup phase.** One clock cycle with `psel` = 1 and `penable` = 0. The master presents `paddr`, `pwrite`, and (for writes) `pwdata` and `pstrb`.
- **Access phase.** Starts in the following cycle with `psel` = 1 and `penable` = 1. It lasts until the cycle in which the slave drives `pready` = 1. The transfer completes at the rising clock edge that ends that cycle.

Between transfers the bus is idle (`psel` = 0, `penable` = 0), or a new setup phase may begin in the cycle immediately after a transfer completes (back-to-back transfer). The slave must behave identically in both cases.

4.2 Address decoding

The slave decodes `paddr` as follows:

- `paddr[7:6]` must be 00 for the address to be mapped. Any other value is an unmapped address.
- `paddr[5:2]` selects one of 16 word registers.
- `paddr[1:0]` are ignored. A byte address inside a word accesses the same register as the word address.

4.3 Write operation

A write to a mapped read/write register updates the register when the transfer completes. For each bit `i` of `pstrb` that is 1, byte lane `i` of the register is replaced by byte lane `i` of `pwdata`. Byte lanes whose strobe is 0 keep their previous value. A write with `pstrb` = 0000 changes nothing and is not an error.

A write to the ID register or to an unmapped address does not modify any register and returns an error (section 4.5), regardless of `pstrb`.

4.4 Read operation

A read of a mapped read/write register returns its current value on `prdata`. A read of the ID register returns `ID_VALUE`. A read of an unmapped address returns 0 and an error response. `pstrb` is ignored on reads.

`prdata` is only guaranteed valid in the cycle where `pready` = 1. The slave may present read data earlier during wait states, but the master must not rely on it. `prdata` is 0 for writes and when no access is in progress.

4.5 Error response

The slave asserts `pslverr` in the same cycle as `pready` (the final cycle of the access phase) when:

1. A write targets the ID register.
2. Any read or write targets an unmapped address.

`pslverr` is 0 in every other cycle, including all setup, idle and wait-state cycles. An error transfer takes the same number of cycles as a normal transfer and does not affect later transfers.

4.6 Wait states

In every transfer the slave holds `pready` low for exactly `WAIT_CYCLES` cycles of the access phase and then drives `pready` high. `pready` is low during setup and idle cycles. The wait count restarts for every transfer.

4.7 Reset

When `presetn` is asserted (low) all read/write registers are cleared to 0 asynchronously and the wait counter is cleared. The ID register is constant and unaffected. After reset is released the slave is idle and ready for a new transfer. Behavior of a transfer that is in progress when reset is asserted is simple: the transfer is abandoned, no register keeps a partial write, and the master is expected to restart.

5. Register Map

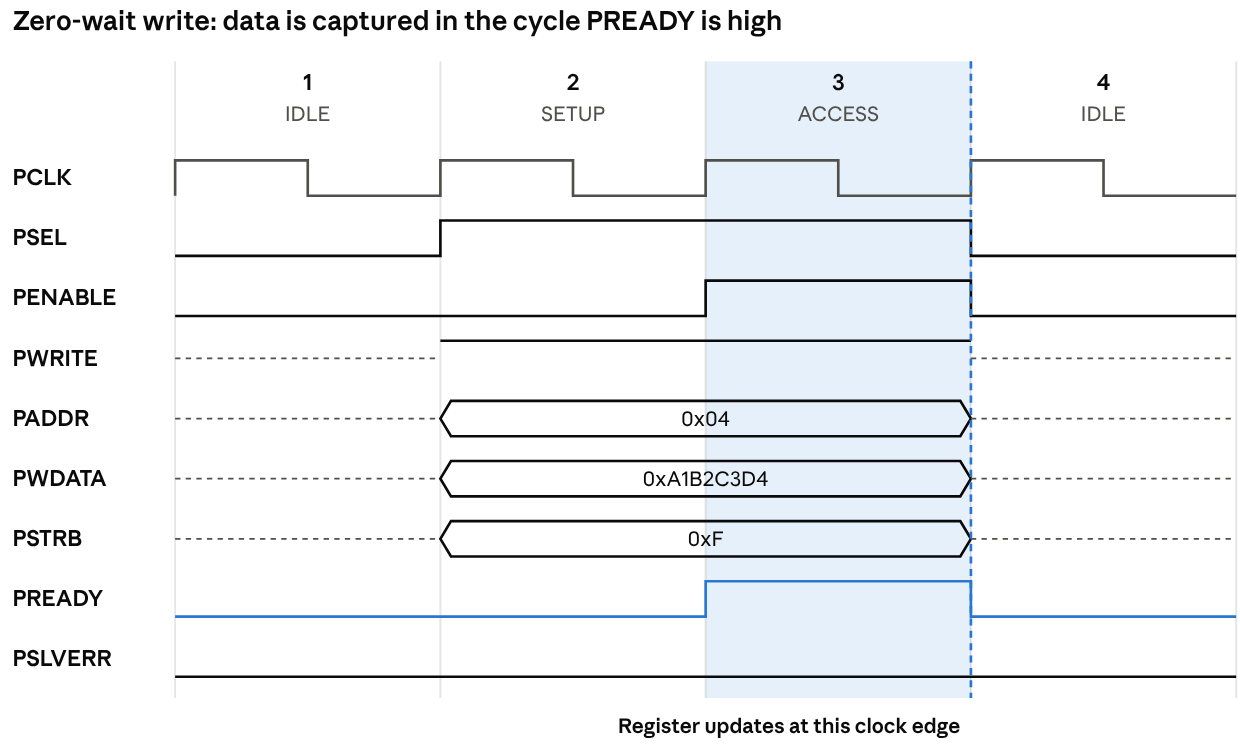
Address	Name	Access	Reset value	Description
0x00	REG0	RW	0x00000000	General-purpose register 0
0x04	REG1	RW	0x00000000	General-purpose register 1
...	...	RW	0x00000000	One register per word address
0x38	REG14	RW	0x00000000	General-purpose register 14

Address	Name	Access	Reset value	Description
0x3C	ID	RO	ID_VALUE	Identification register. Writes return PSLVERR and are ignored.
0x40 to 0xFF	(unmapped)	n/a	n/a	Reads return 0 with PSLVERR. Writes are ignored with PSLVERR.

6. Timing Examples

6.1 Write, WAIT_CYCLES = 0

Cycle	psel	penable	pready	Phase
1	0	0	0	Idle
2	1	0	0	Setup (address, data, strobes presented)
3	1	1	1	Access, transfer completes, register updated at the end of this cycle

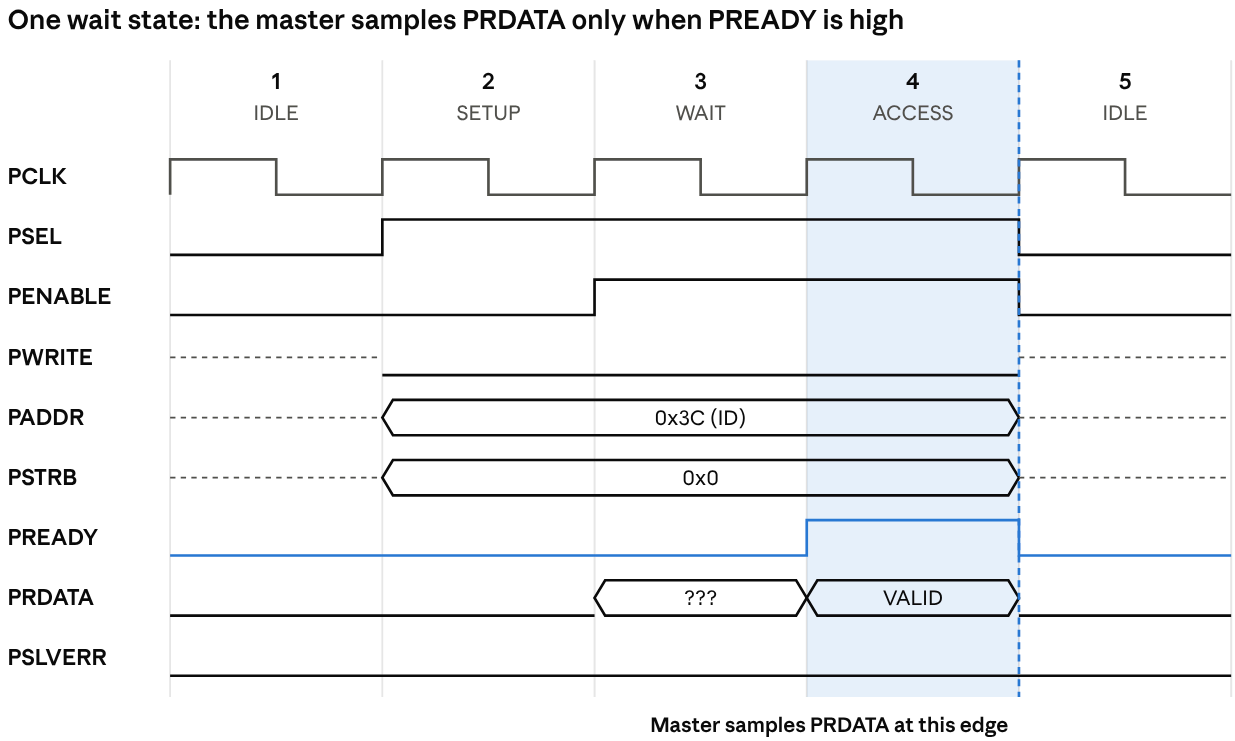


write, 0 wait states · timing waveform

The register is updated at the clock edge that ends cycle 3, the cycle where PREADY is high. PRDATA stays 0 and is not drawn.

6.2 Read, WAIT_CYCLES = 1

Cycle	psel	penable	pready	prdata	Phase
1	1	0	0	0	Setup
2	1	1	0	may be valid	Access, wait state
3	1	1	1	valid	Access, transfer completes, master samples prdata

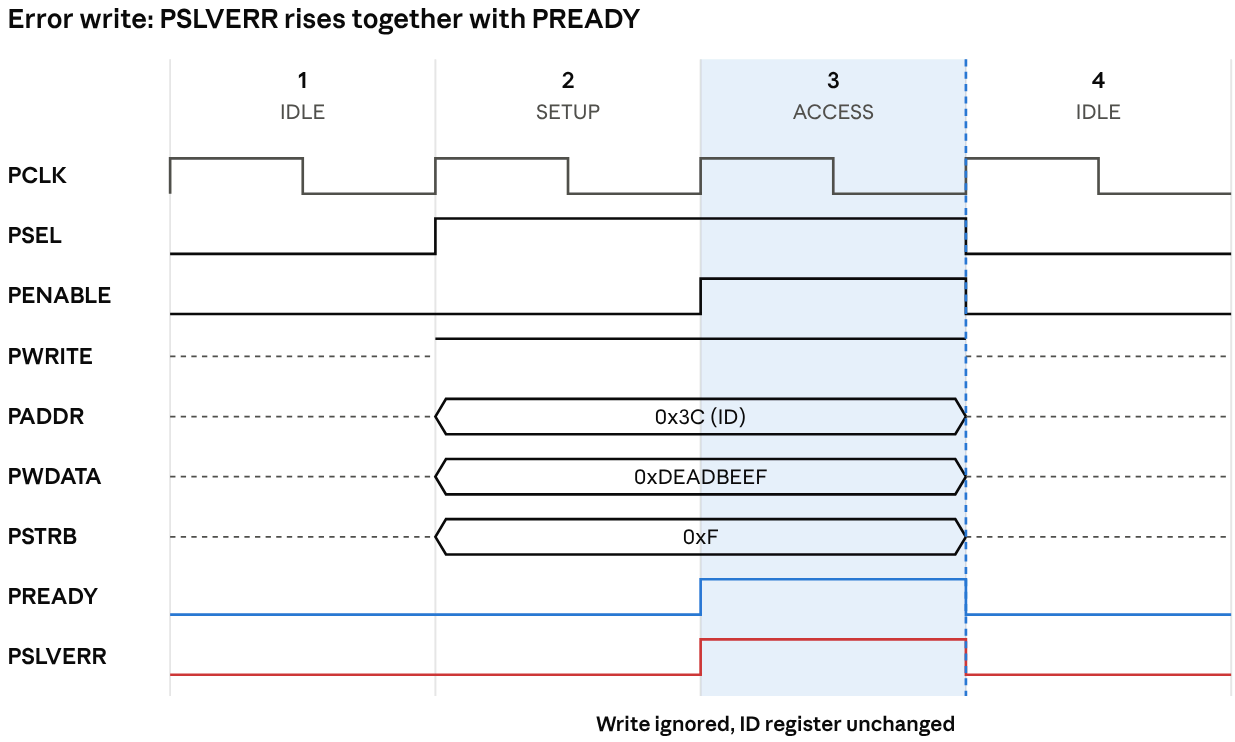


read, 1 wait state · timing waveform

The slave inserts one wait state in cycle 3. PRDATA may already show data during the wait state, but the master must sample it only in cycle 4, where PREADY is high.

6.3 Write to the ID register, WAIT_CYCLES = 0

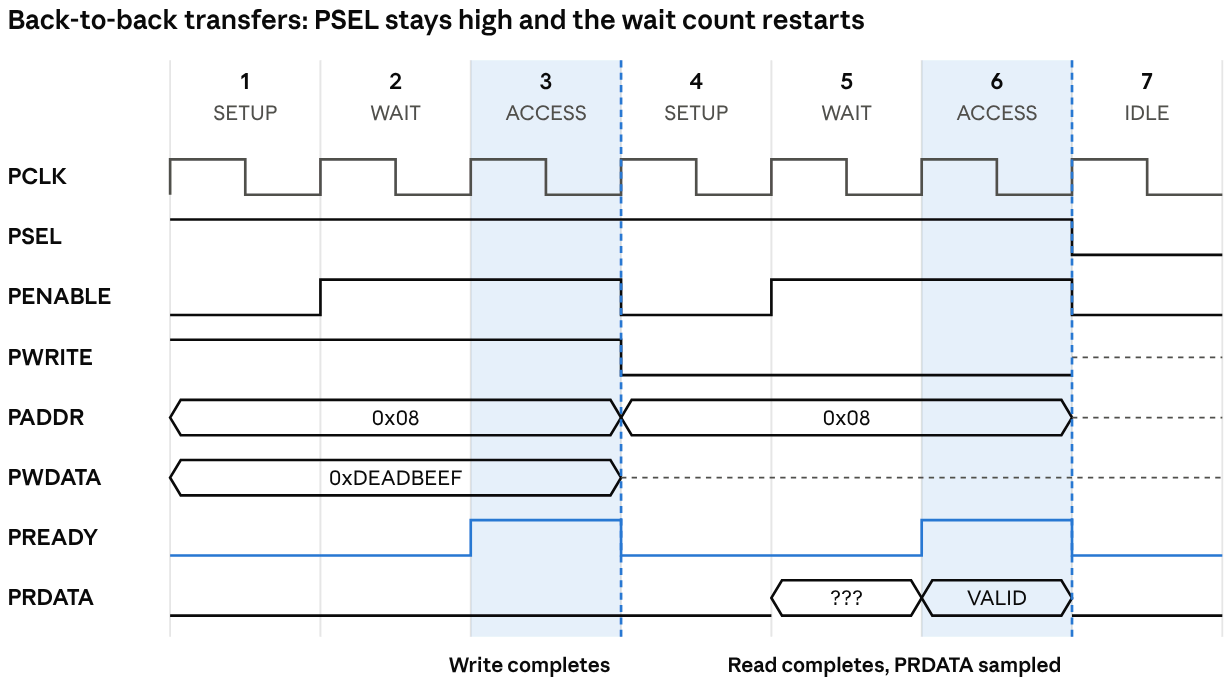
Cycle	psel	penable	pready	pslverr	Phase
1	1	0	0	0	Setup
2	1	1	1	1	Access, completes with error, ID unchanged



write to ID register, error response · timing waveform

PSLVERR rises together with PREADY in cycle 3 and drops in cycle 4. The write is ignored and the ID register keeps its value.

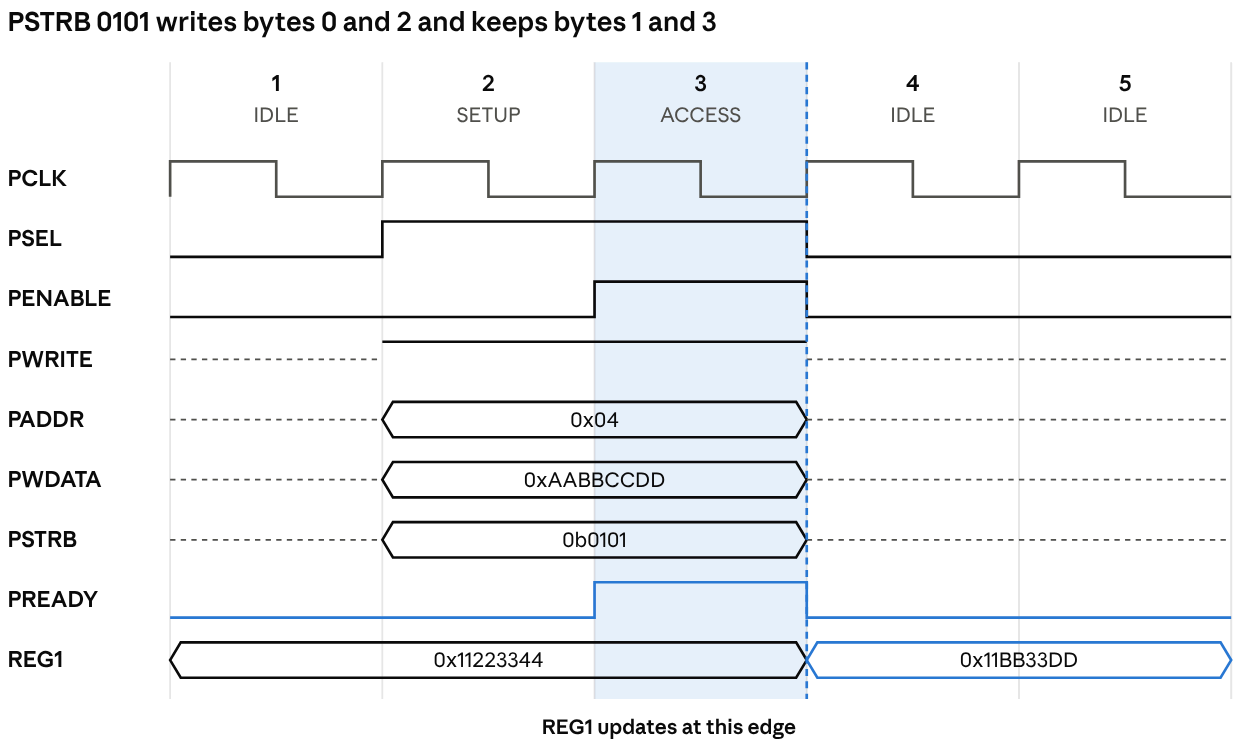
6.4 Back-to-back write then read, WAIT_CYCLES = 1



back-to-back write then read · timing waveform

The master writes 0xDEADBEEF to address 0x08, then reads the same address. The second setup phase starts in the cycle right after the first transfer completes, and PSEL stays high between the two. The wait counter restarts, so the read also gets one wait state and returns 0xDEADBEEF.

6.5 Partial write with PSTRB, WAIT_CYCLES = 0



partial write with PSTRB 0101 · timing waveform

Only byte lanes 0 and 2 (PSTRB = 0101) are written. Bytes 1 and 3 keep their old values, so REG1 at address 0x04 changes from 0x11223344 to 0x11BB33DD at the end of cycle 3.

7. Assumptions on the APB Master

The design relies on the master to follow the APB protocol:

- `paddr`, `pwrite`, `pwdata`, `pstrb` and `psel` stay stable from the setup phase until the transfer completes.
- Every transfer starts with a setup phase. `penable` is never asserted without `psel`, and the access phase is never entered without a preceding setup phase.
- `pstrb` is driven to 0000 for reads.
- The master does not abort a transfer once started, except by asserting reset.
- `presetn` is asserted for at least one clock period.

Behavior when these assumptions are violated is undefined and is not verified.

8. Items Left Undefined

- Value of `prdata` during wait states of a read (see 4.4).
- Behavior for `WAIT_CYCLES` greater than 255.
- Power, area, and timing closure. They are out of scope for this specification.
- Clock-domain crossing. The block is single-clock.

9. Revision History

Version	Description
1.0	First release for verification.